CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING (C-DAC)

THIRUVANANTHAPURAM, KERALA

A MAJOR PROJECT REPORT ON

"Vulnerability Assessment and Penetration Testing (VAPT) on Altoro Mutual (testfire.net)"

SUBMITTED TOWARDS THE

PGCP-CSF February 2026

SUBMITTED BY : Group Number 02

STUDENT NAME	PRN NO.
ABHISHEK SRIVASTAVA	260260940002
AKSHAY SATISH KALBHOR	260260940005
CHOUDHARI PRASAD RAJARAM	260260940014
DEEPAK ASHOK KALEWAR	260260940016
HARSHAL JITENDRA KOLHE	260260940018
SATYAM BHAGWAN JADHAV	260260940042
SAURABH MADHUKAR KADUKAR	260260940043
SHANTANU SHIVRAJ SUPEKAR	260260940045

TABLE OF CONTENTS

Executive Summary / Abstract

Web applications are increasingly used to provide critical services and process sensitive information, making application security an important component of modern cybersecurity. Vulnerabilities in web applications can allow unauthorized users to access sensitive information, manipulate application functionality, execute malicious scripts, bypass authentication mechanisms, or compromise the confidentiality, integrity, and availability of application resources.

The objective of this major project is to perform a controlled Vulnerability Assessment and Penetration Testing (VAPT) exercise against the Altoro Mutual web application hosted on testfire.net. The assessment was conducted in a controlled laboratory environment using Kali Linux, Burp Suite, Nmap, and SQLmap as the primary security testing tools.

The assessment consisted of reconnaissance, network enumeration, service identification, SSL/TLS security assessment, HTTP request analysis, input validation testing, authentication testing, and vulnerability validation. Multiple application-level and network-level security weaknesses were identified, including Reflected Cross-Site Scripting (XSS), Stored Cross-Site Scripting, HTML Injection, a CSRF protection weakness requiring further validation, SQL Injection, SQL Injection-based authentication bypass, exposed network services, service/version disclosure, and weak TLS configuration.

Burp Suite was used to intercept and analyze HTTP requests and responses and to validate web application vulnerabilities. Nmap was used for host discovery, operating system detection, service/version enumeration, and SSL/TLS cipher assessment. SQL Injection testing demonstrated database errors and, in the documented authentication-bypass test, authentication without valid credentials.

The findings were documented with proof-of-concept observations, security impacts, risk levels, and recommended remediation measures. The project demonstrates the importance of systematic security testing, secure input handling, strong authentication controls, appropriate session protection, secure network configuration, and continuous vulnerability assessment.

Keywords: Vulnerability Assessment, Penetration Testing, VAPT, Web Application Security, Altoro Mutual, Testfire.net, Burp Suite, Nmap, SQL Injection, Cross-Site Scripting, CSRF, SSL/TLS Security.

1. Comprehensive Introduction & VAPT Methodology

Web applications are exposed to a wide range of security threats because they accept and process user-controlled input, maintain authentication sessions, interact with databases, and communicate over public networks. A vulnerability in any of these components can provide an attacker with an opportunity to compromise application functionality or sensitive information.

Vulnerability Assessment and Penetration Testing (VAPT) is a structured security testing methodology used to identify security weaknesses and validate their practical impact. Vulnerability assessment focuses primarily on discovering and identifying weaknesses, while penetration testing involves controlled attempts to validate whether identified weaknesses can actually be exploited.

The present project focuses on the security assessment of the Altoro Mutual demonstration banking application hosted on testfire.net. The application is designed as a deliberately vulnerable environment for security testing and training.

The assessment followed a systematic workflow consisting of reconnaissance, enumeration, vulnerability identification, validation where appropriate, impact assessment, and remediation recommendation.

1.1 Vulnerability Assessment and Penetration Testing

The VAPT process combines automated and manual security testing techniques. The assessment performed during this project can be broadly divided into reconnaissance, network enumeration, service identification, web application mapping, input validation testing, authentication and authorization testing, vulnerability validation, risk analysis, and remediation recommendation.

1. Reconnaissance
2. Network Enumeration
3. Service Identification
4. Web Application Mapping
5. Input Validation Testing
6. Authentication and Authorization Testing
7. Vulnerability Validation
8. Risk and Impact Analysis
9. Remediation Recommendation

1.2 Objectives of the Project

10. Perform security reconnaissance against the target application.
11. Identify exposed network ports and services.
12. Identify service and technology information through enumeration.
13. Analyze HTTP request and response behavior.
14. Test user-controlled input for injection vulnerabilities.
15. Identify Cross-Site Scripting vulnerabilities.
16. Test for HTML Injection.
17. Assess Cross-Site Request Forgery protection.
18. Evaluate the application's exposure to Server-Side Request Forgery.
19. Test authentication functionality for SQL Injection.
20. Validate SQL Injection-based authentication bypass.
21. Assess SSL/TLS configuration.
22. Document proof-of-concept evidence.
23. Evaluate the security impact of identified vulnerabilities.

24. Recommend appropriate remediation measures.

1.3 Scope of the Assessment

The assessment was restricted to the Altoro Mutual / testfire.net web application and the associated network services observable during the controlled testing process.

- Network reconnaissance and host discovery
- Open-port identification
- Service and version enumeration
- Operating system fingerprinting
- SSL/TLS configuration assessment
- HTTP request/response analysis
- Input validation
- Cross-Site Scripting
- HTML Injection
- CSRF assessment
- SSRF attack-surface assessment
- SQL Injection testing
- Authentication bypass testing

1.4 Target Application Overview

The target application is Altoro Mutual, a deliberately vulnerable demonstration banking application available through testfire.net. The application provides banking-related functionality such as authentication and fund transfer operations. These functions provide a useful environment for studying common web application security weaknesses.

1.5 Tools & Technologies Used

Tool / Platform	**Primary Purpose**
Kali Linux	Primary penetration-testing operating environment.
Burp Suite	HTTP interception, request modification, Repeater testing, session and parameter analysis.
Nmap	Host discovery, port scanning, OS detection, service/version detection and SSL/TLS enumeration.
SQLmap	SQL Injection detection and validation toolkit used as part of the project toolkit.

2. Phase I: Reconnaissance & Network Security Assessment

The first phase focused on identifying the target system and understanding its exposed network attack surface. Network reconnaissance helps security testers identify active hosts, open ports, running services, operating system characteristics, and cryptographic configuration weaknesses.

Target IP used during the documented network assessment: 65.61.137.117. Nmap 7.95 was used.

2.1 Host Discovery Using Nmap

Objective: Determine whether the target system was reachable over the network before performing detailed enumeration.

```
nmap -sn 65.61.137.117
```

The -sn option performs host discovery without a conventional port scan. The target host responded successfully, an ICMP Echo Reply was received, and the host was considered reachable.

Risk Level: Informational.

Impact: Host discovery does not directly compromise the target, but it helps attackers identify active systems and perform network mapping.

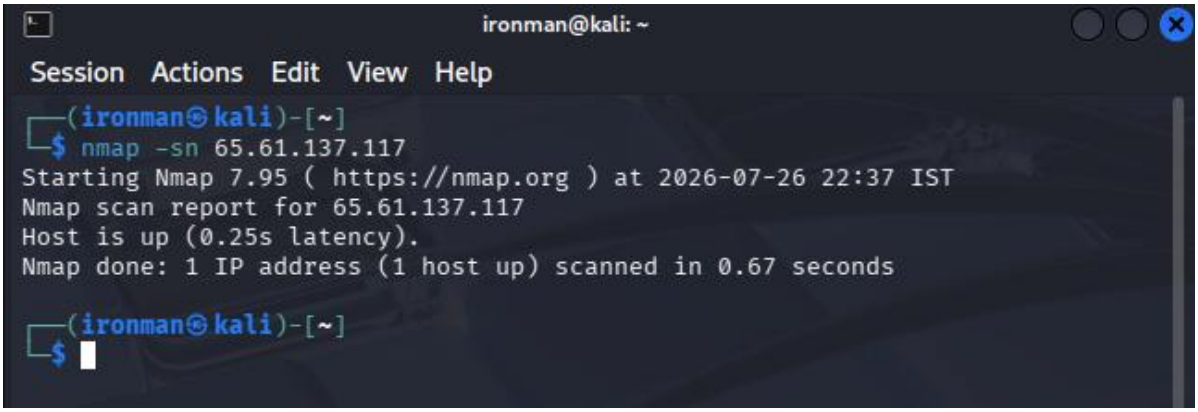

Figure 2.1 — Nmap Host Discovery Result

Information / Observation: The screenshot shows the Nmap host-discovery execution in the Kali Linux terminal. It provides evidence that the target responded during the reconnaissance phase and was available for subsequent enumeration.

Remediation: Restrict unnecessary ICMP traffic, apply firewall rules, permit ICMP from trusted administrative networks where operationally possible, and monitor abnormal scanning activity.

2.2 Operating System Detection

```
nmap -O 65.61.137.117
```

The assessment identified ports 80 (HTTP), 443 (HTTPS), and 8080 (HTTP Alternate) as open, while port 8443 was reported closed. Nmap could not accurately determine the OS because of filtered UDP responses, but the target appeared to be Linux-based.

Risk Level: Low.

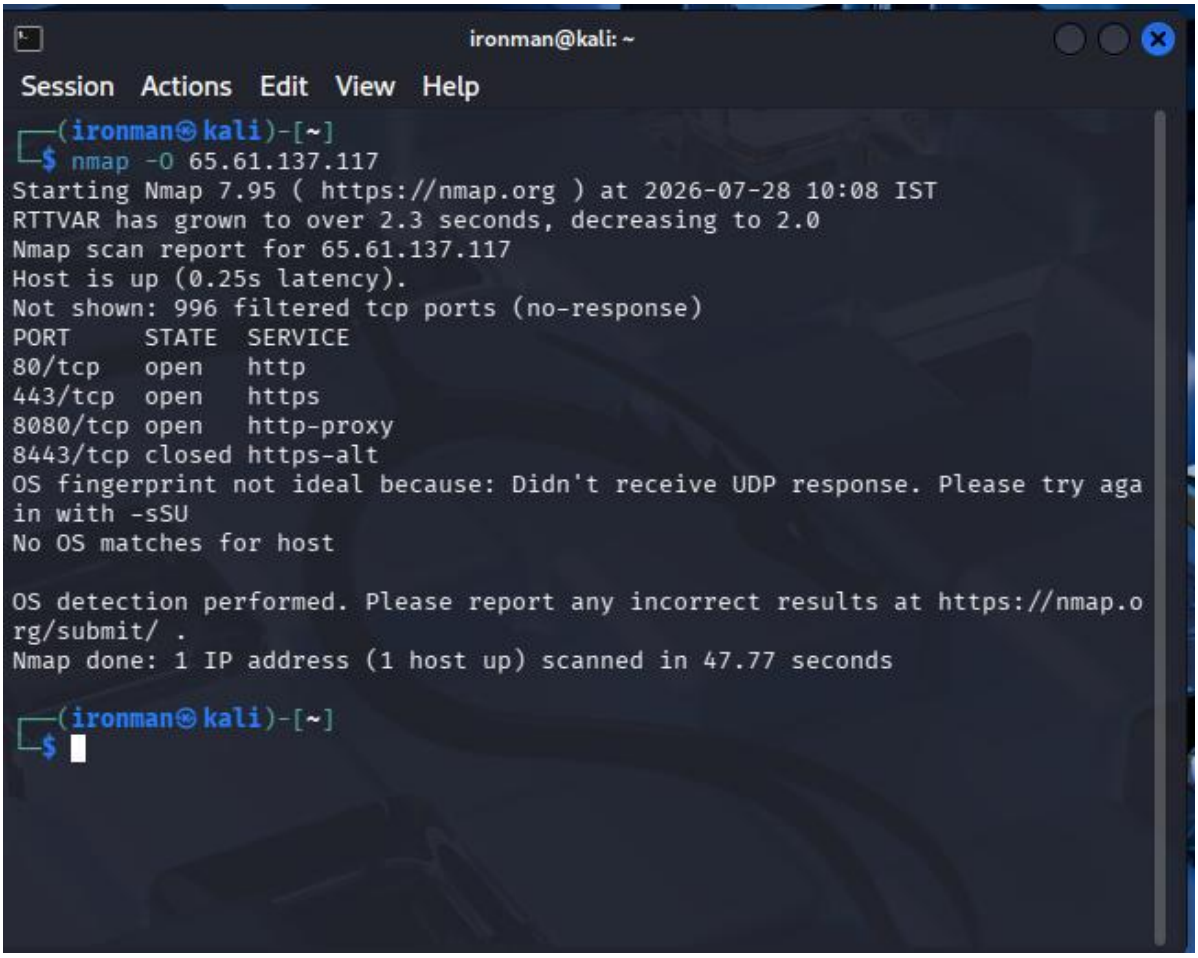

Figure 2.2 — Nmap Operating System Detection

Information / Observation: The terminal output shows the OS-detection scan and the discovered network characteristics. The result provides a fingerprinting view of the target that can assist further security testing.

Remediation: Close unnecessary ports, restrict externally accessible services, disable unused network services, apply regular security updates, implement network segmentation, and monitor unauthorized reconnaissance.

2.3 Service & Version Detection

```
nmap -sV -A 65.61.137.117
```

The service enumeration identified Apache Tomcat and Apache Coyote JSP Engine, along with HTTP on port 80, HTTPS on port 443, and HTTP Alternate on port 8080. Banner and certificate information also disclosed application and server details.

Risk Level: Medium.

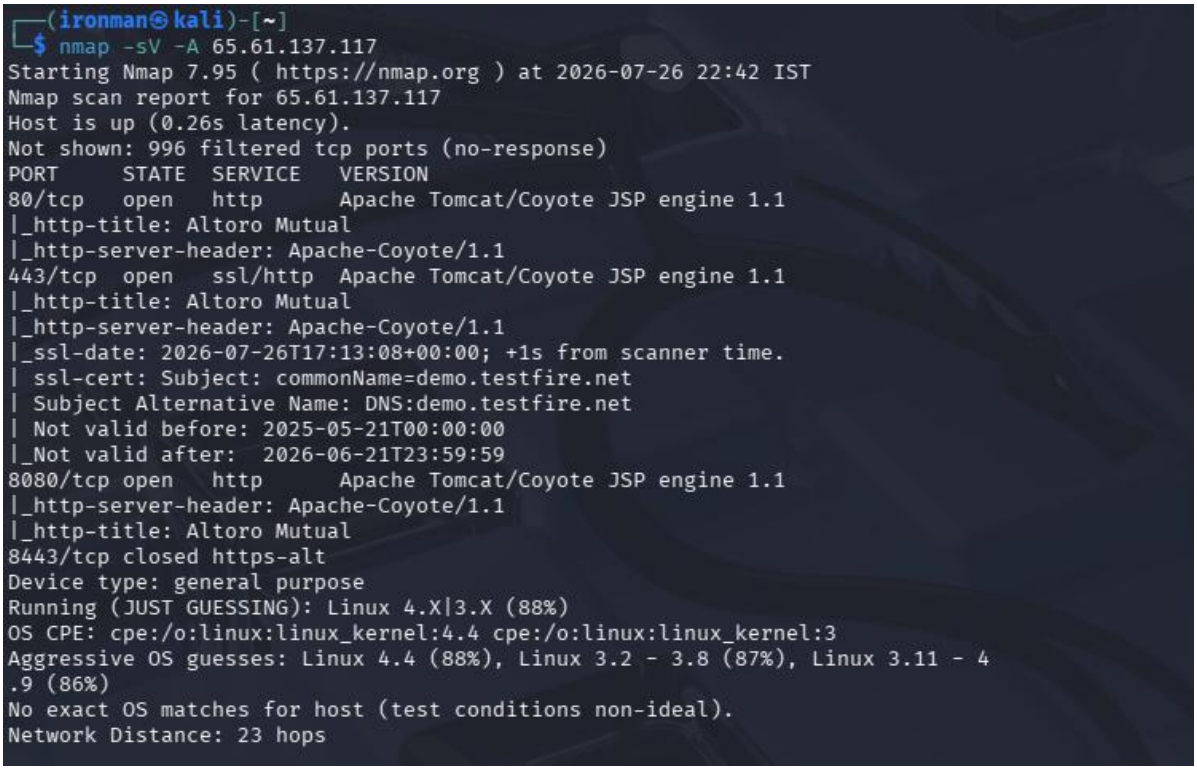

Figure 2.3 — Nmap Service and Version Detection

Information / Observation: The screenshot provides service/version enumeration evidence. Apache Tomcat/Coyote and exposed HTTP/HTTPS services are visible in the Nmap output. Such information can help attackers identify technologies and search for known vulnerabilities.

Remediation: Keep server software updated, suppress unnecessary banners, minimize information disclosure through HTTP headers, replace demonstration certificates in production, perform periodic vulnerability assessments, and apply vendor patches promptly.

2.4 SSL/TLS Security Audit

```
nmap --script ssl-enum-ciphers -p443 65.61.137.117
```

The SSL/TLS enumeration identified TLS 1.0, TLS 1.1, weak DH1024 key exchange support, and CBC cipher suites. Modern protocols were also supported.

Risk Level: Medium.

Figure 2.4 — SSL/TLS Cipher Enumeration

Information / Observation: The screenshot shows the SSL/TLS enumeration output from Nmap. It provides evidence of the supported cryptographic protocols and cipher-related configuration observed during the assessment.

Remediation: Disable TLS 1.0 and TLS 1.1, enable TLS 1.2/1.3, replace weak DH1024 parameters with stronger parameters, prefer modern ECDHE key exchange, disable weak cipher suites where possible, and regularly test TLS configuration.

2.5 Network Security Assessment Summary

Test	Result	Risk
Host Discovery	Target reachable	Informational
OS Detection	Linux-based OS indicated	Low
Service Detection	Apache Tomcat/Coyote identified	Medium
Port Enumeration	80, 443, 8080 open; 8443 closed	Low/Medium
SSL/TLS Audit	TLS 1.0/1.1 and weak DH1024 identified	Medium

3. Phase II: Web Application Security Assessment

The second phase focused on the web application's functionality and handling of user-controlled input. Burp Suite was used extensively to intercept and modify requests, inspect parameters, analyze sessions, and validate security controls.

3.1 Reflected Cross-Site Scripting

Vulnerability: Reflected Cross-Site Scripting (XSS)

Affected URL: https://demo.testfire.net/index.jsp

Objective: Determine whether user-controlled search input was reflected into the application's response without appropriate output encoding.

```
<script>alert('XSS test')</script>
```

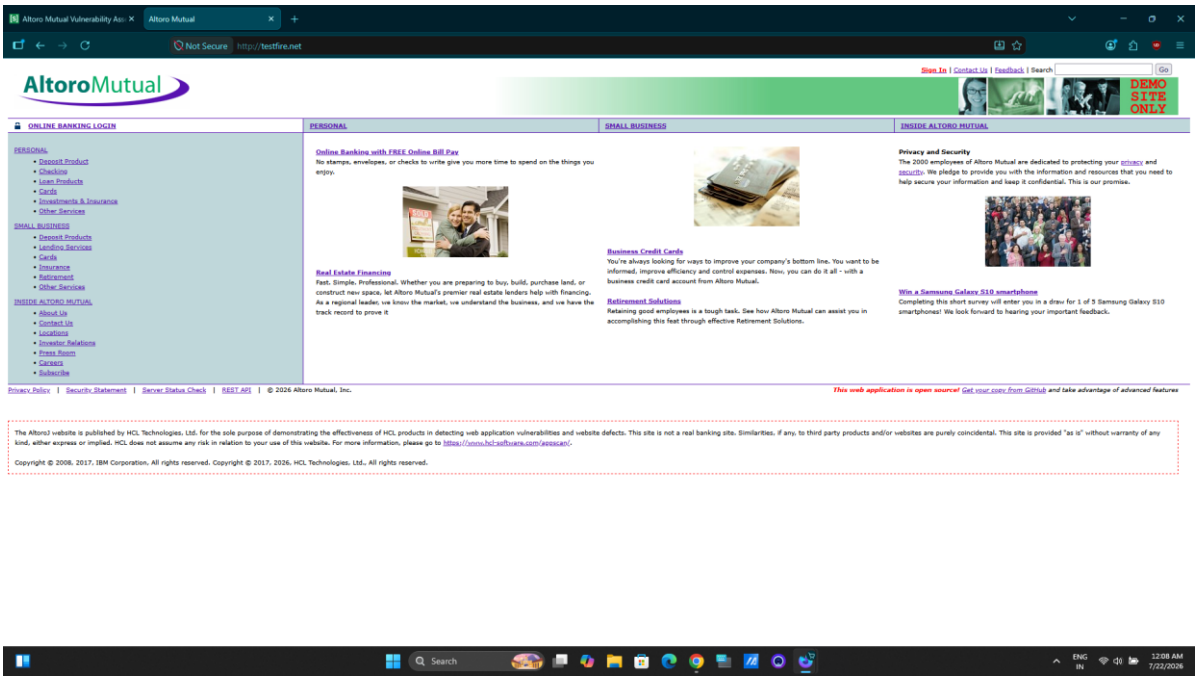

Figure 3.1 — Reflected XSS Test: Search Functionality

Information / Observation: The screenshot shows the Altoro Mutual search functionality used for the reflected XSS test. The search parameter was selected because user-controlled input is returned in the application's response.

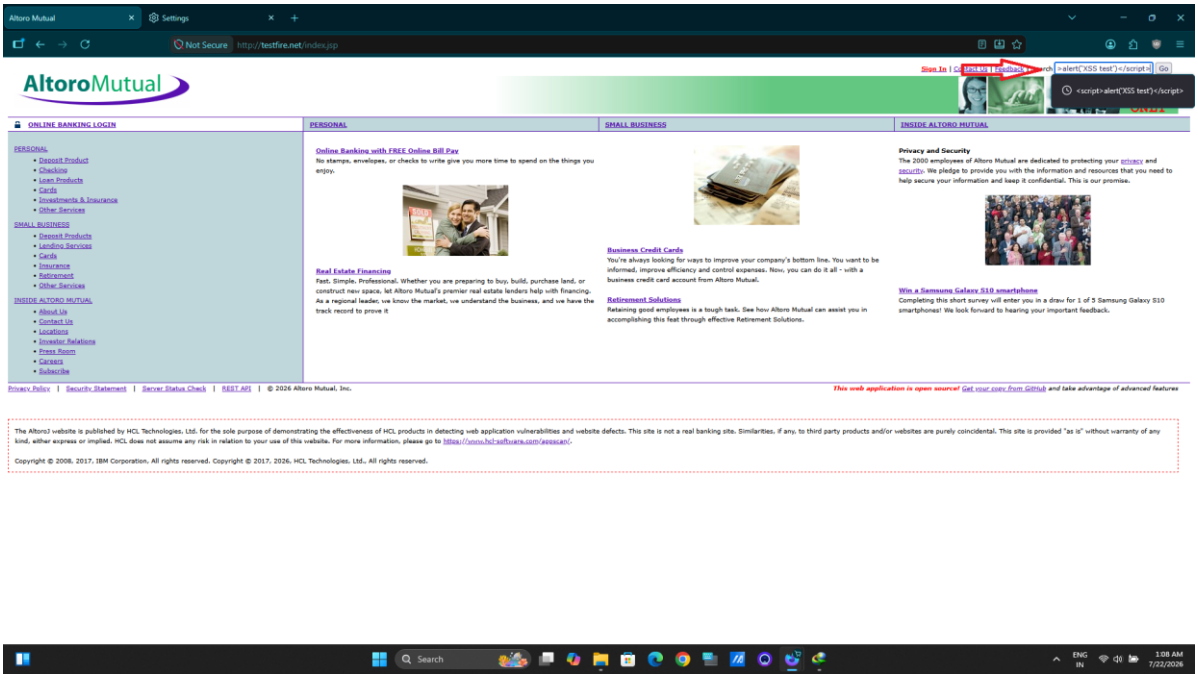

Figure 3.2 — Reflected XSS Payload Submission

Information / Observation: The screenshot documents the reflected-XSS testing stage. The injected script was supplied through the search functionality and was reflected rather than permanently stored.

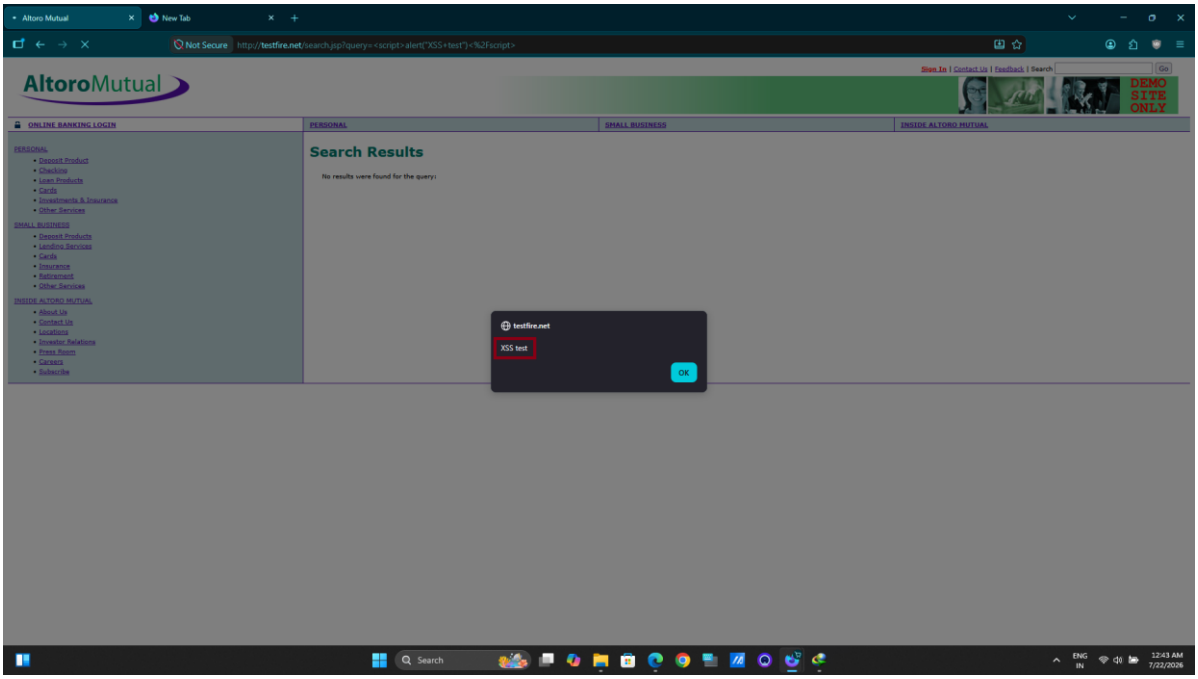

Figure 3.3 — Reflected XSS Execution Result

Information / Observation: The screenshot shows the browser response after the XSS payload was processed, providing visual evidence for the reflected behavior.

```
<script>alert(document.cookie)</script>
```

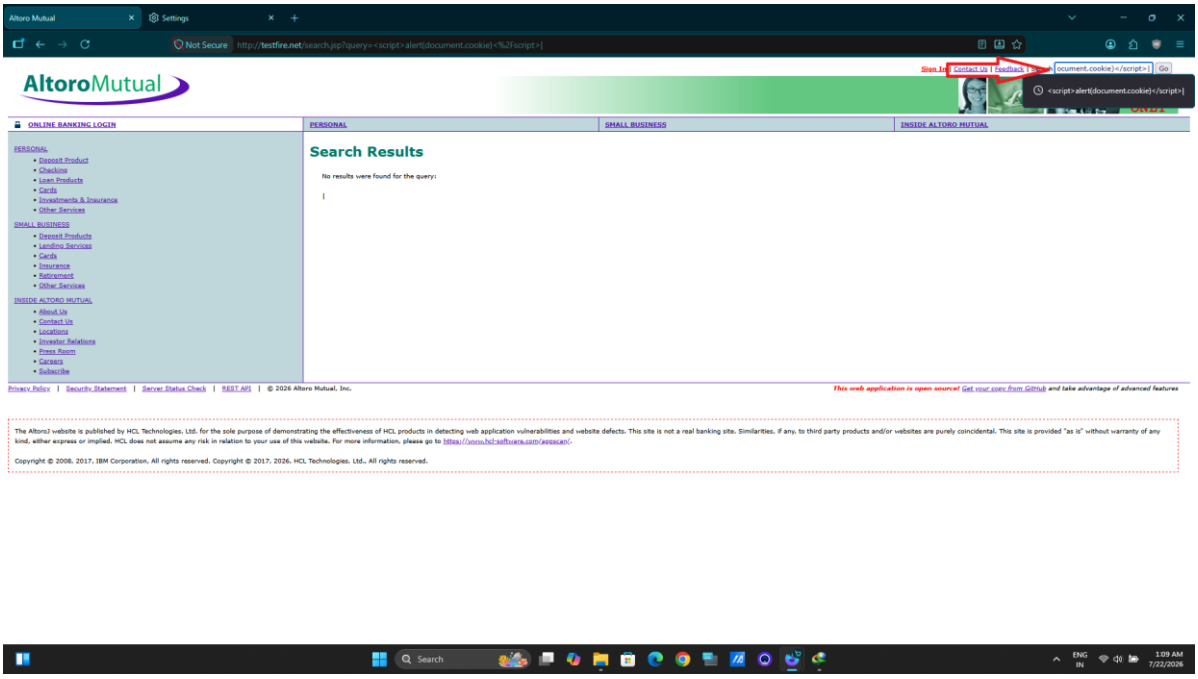

Figure 3.4 — Cookie Access Test Payload

Information / Observation: This screenshot documents the additional client-side script test used to examine whether browser cookie information could be accessed by injected JavaScript.

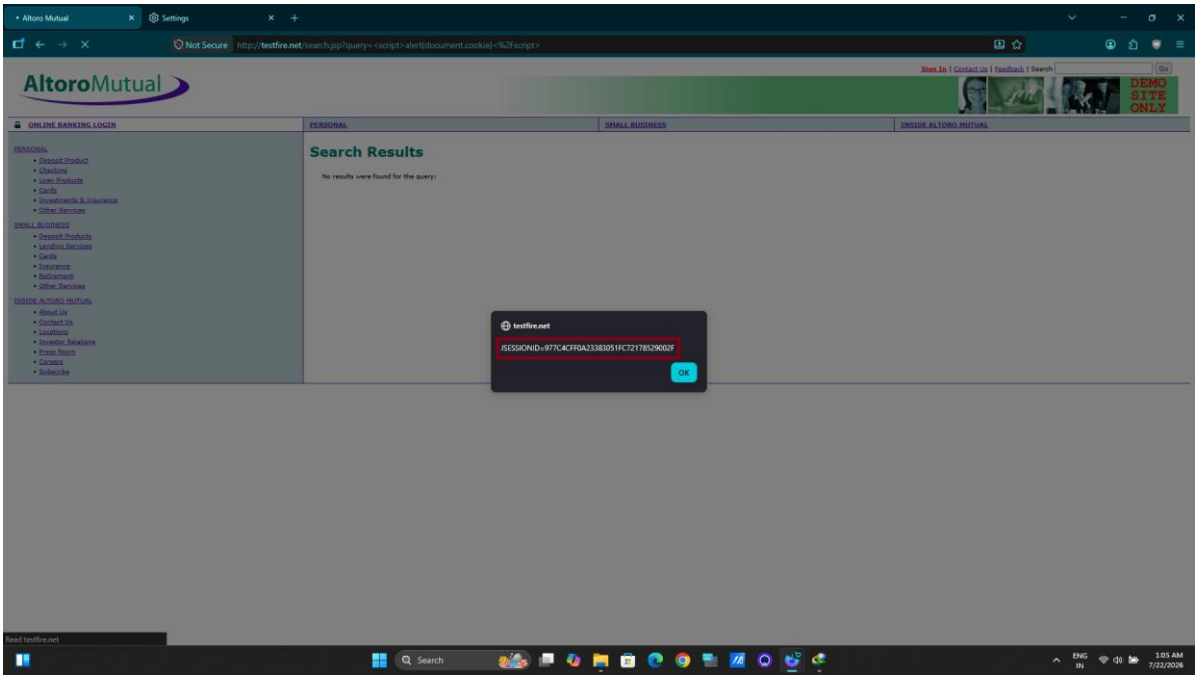

Figure 3.5 — Cookie Test Result

Information / Observation: The response demonstrates the result of the client-side cookie-access test. It reinforces the need for secure cookie attributes such as HttpOnly and Secure.

Impact: Reflected XSS can permit malicious JavaScript execution in a victim browser and may lead to session compromise, information disclosure, phishing, page manipulation, and other client-side attacks.

Risk Level: High.

- Validate user input on the server side.
- Apply context-aware output encoding.
- Implement a strong Content Security Policy.
- Enable HttpOnly for cookies that do not require JavaScript access.
- Use Secure cookies over HTTPS.
- Perform regular XSS security testing.

3.2 HTML Injection

Vulnerability: HTML Injection

Affected URL: https://demo.testfire.net/search.jsp

```
<h1>Testing HTML</h1>
```

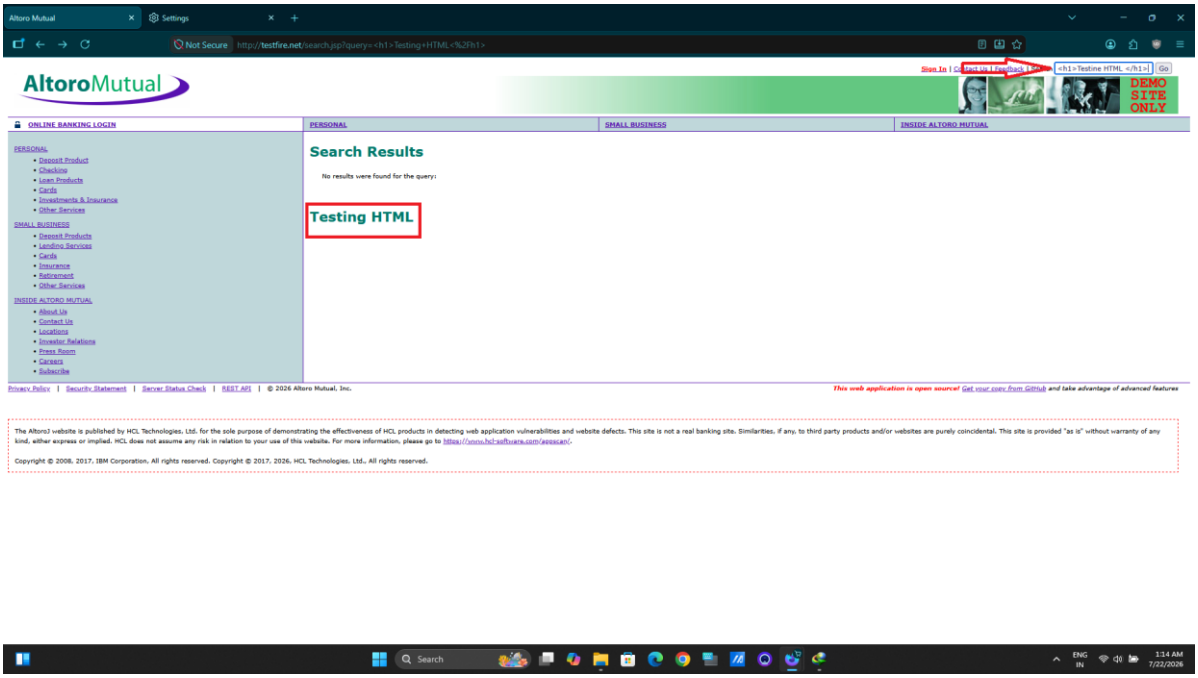

Figure 3.6 — HTML Injection Payload

Information / Observation: The screenshot documents the search functionality where HTML markup was supplied as user-controlled input.

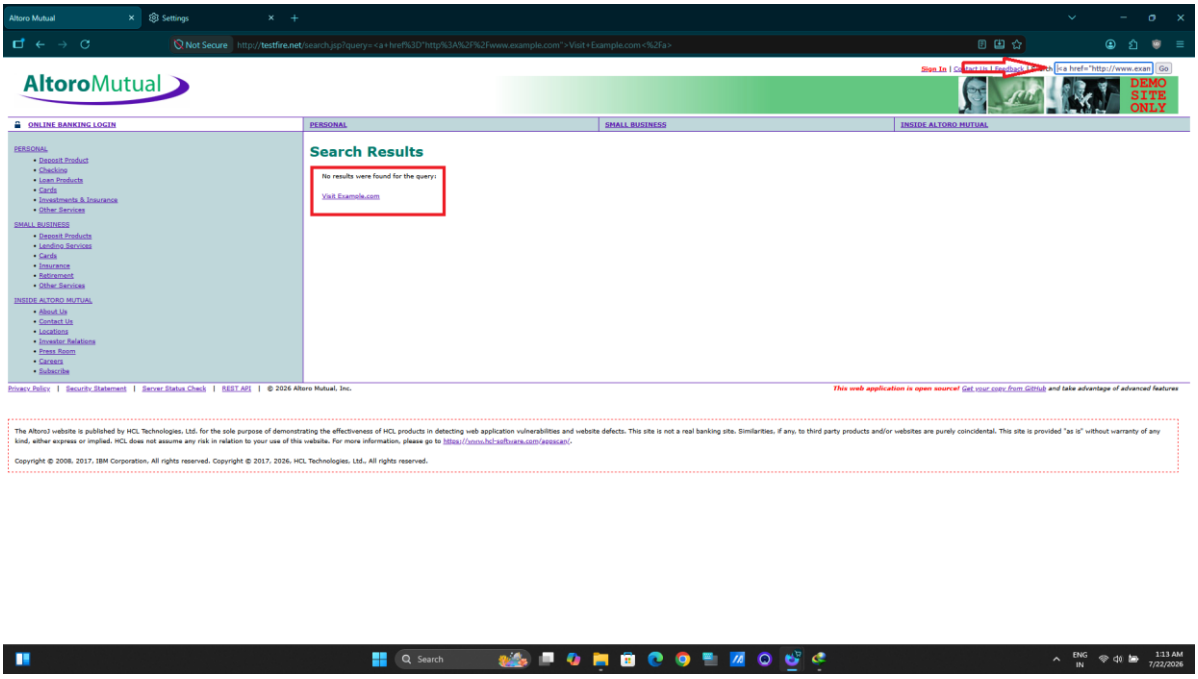

Figure 3.7 — HTML Injection Result

Information / Observation: The screenshot shows the rendered application response associated with the HTML-injection test. It demonstrates why raw user-controlled markup should not be rendered without appropriate validation and output encoding.

An attacker could also construct malicious HTML links, for example: Visit Example.com.

Impact: HTML Injection can modify page content, facilitate phishing, create attacker-controlled links within a trusted domain, and mislead users.

Risk Level: Medium.

- Validate input on the server side.
- Encode user-controlled output.
- Reject unnecessary HTML markup.
- Use allowlists when specific HTML is genuinely required.
- Avoid rendering raw user-controlled HTML.

3.3 Stored Cross-Site Scripting

Vulnerability: Stored Cross-Site Scripting

Affected URL: http://testfire.net/feedback.jsp

The feedback form was tested through Burp Suite. Input fields were intercepted and modified using Burp Suite Repeater.

```
"><script>alert('Stored_XSS_test')</script>
```

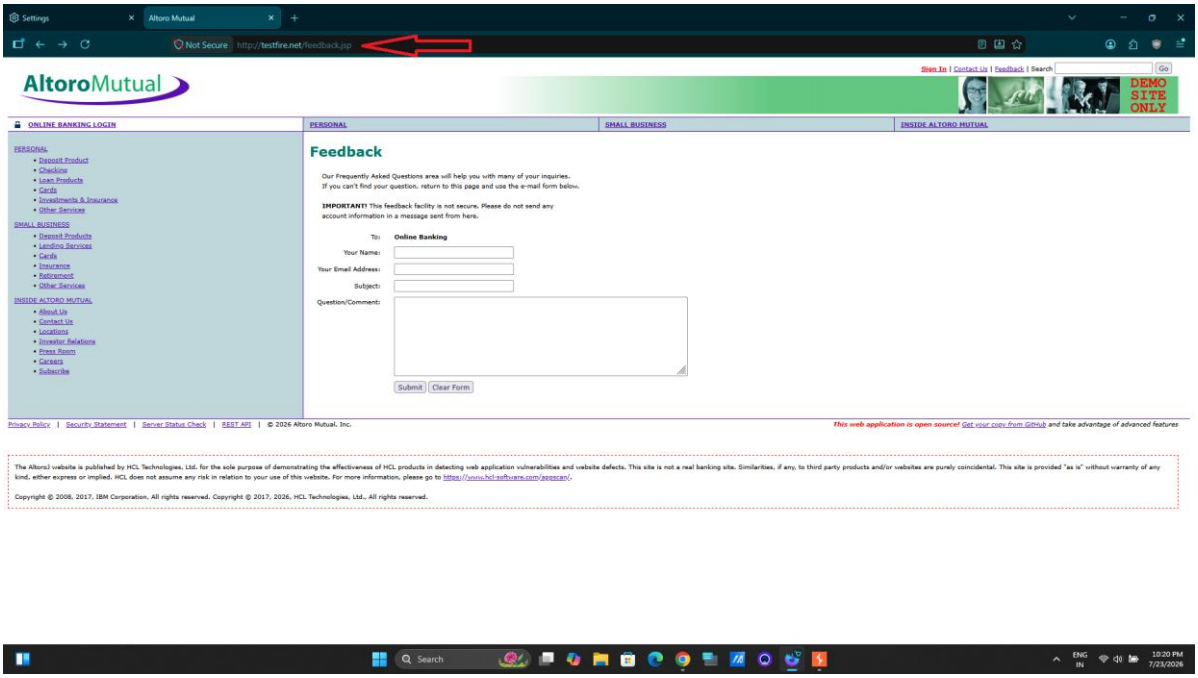

Figure 3.8 — Stored XSS Testing Through Feedback Form

Information / Observation: The screenshot shows the feedback functionality selected for stored-XSS testing.

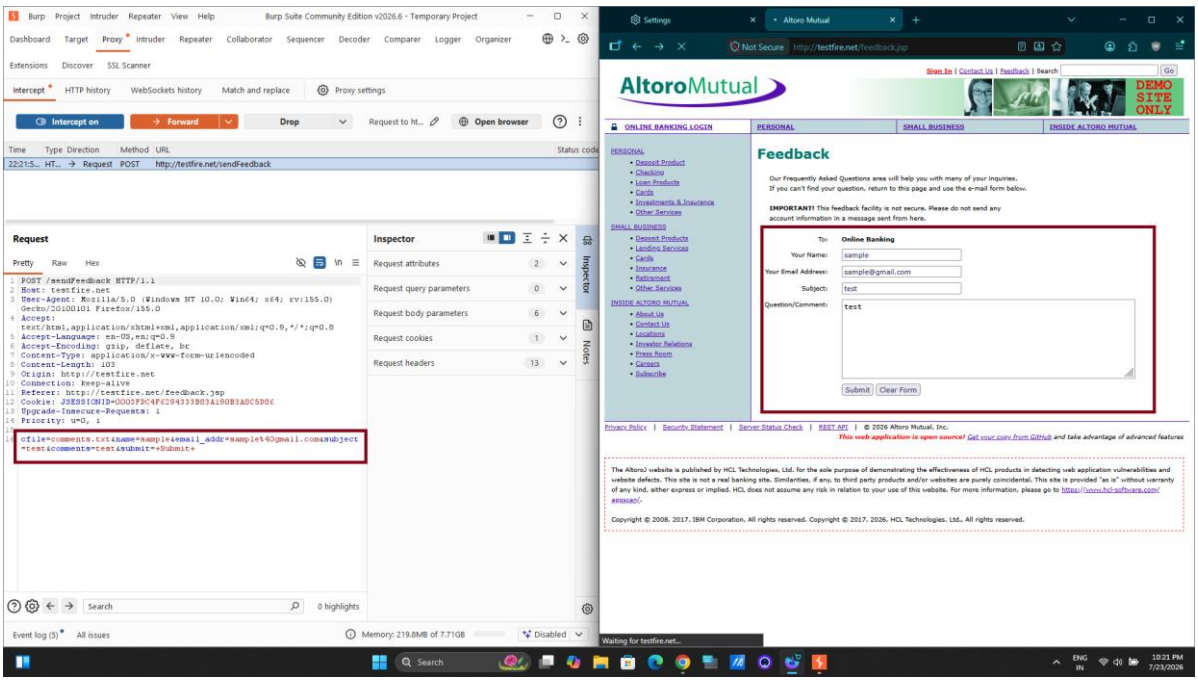

Figure 3.9 — Stored XSS Request Analysis in Burp Suite

Information / Observation: The screenshot documents the intercepted request and testing workflow used to manipulate the feedback input.

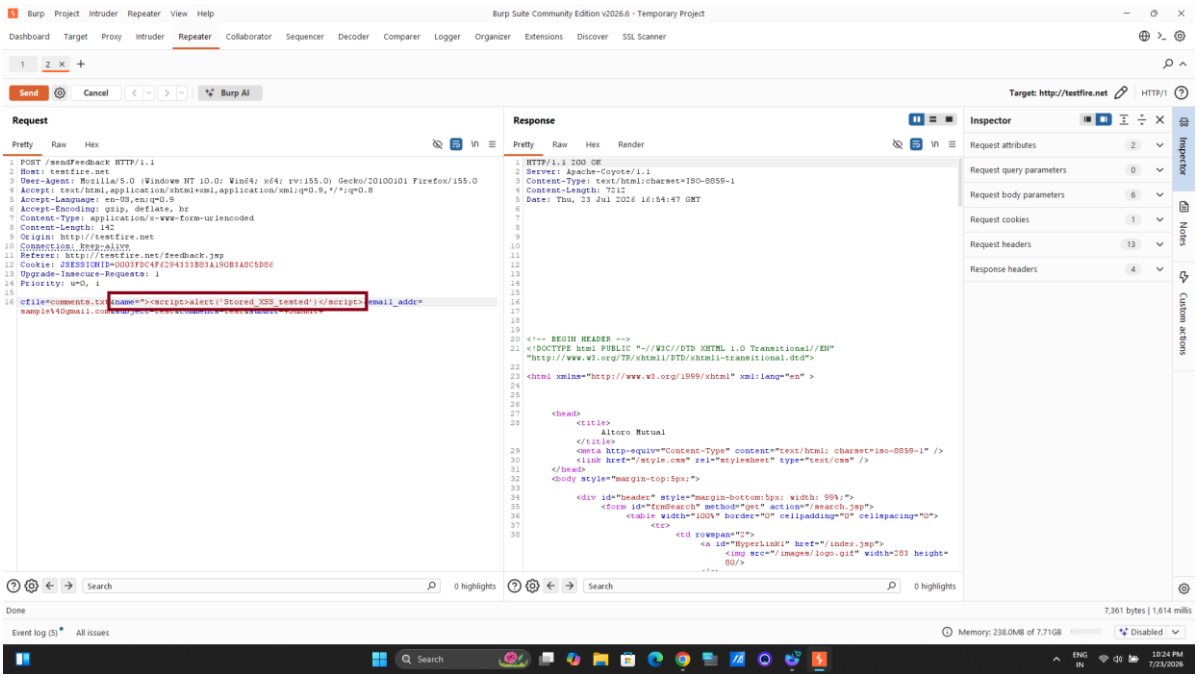

Figure 3.10 — Stored XSS Payload in Request

Information / Observation: The screenshot shows the crafted JavaScript payload being tested against the vulnerable input field.

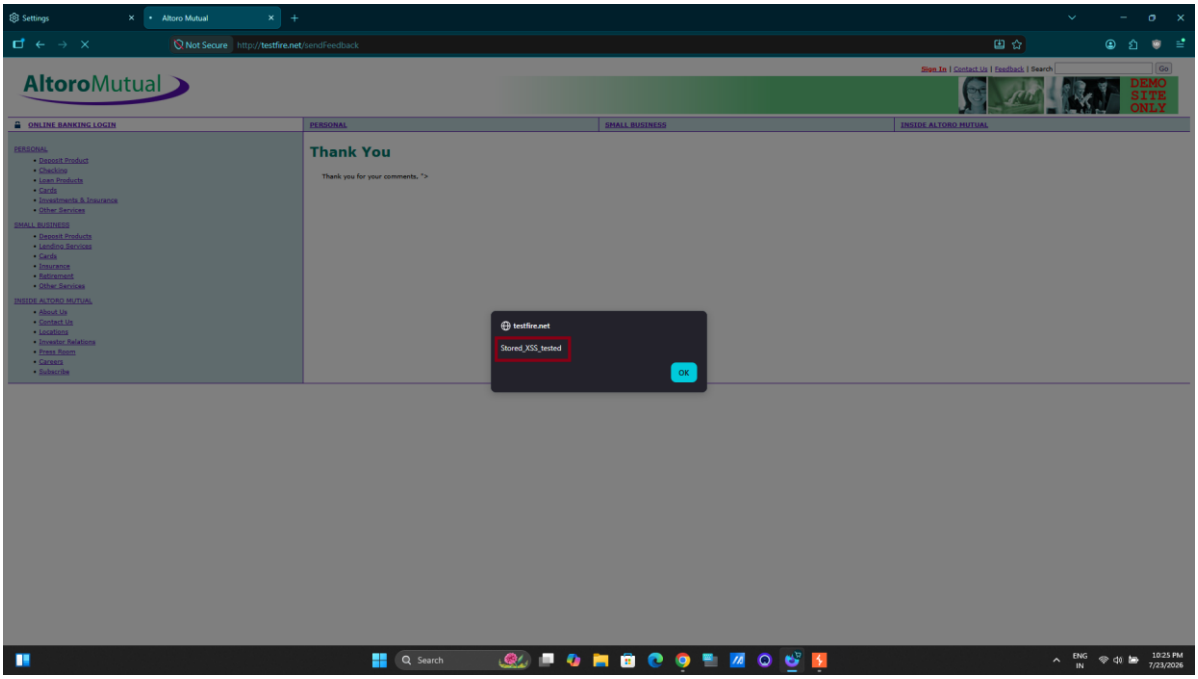

Figure 3.11 — Stored XSS Result

Information / Observation: The screenshot provides evidence of the resulting application behavior after the stored-XSS payload was submitted.

Impact: Successful exploitation may allow malicious JavaScript execution, session compromise where protections are inadequate, page defacement, phishing, information theft, and further client-side attacks.

Risk Level: High.

- Validate input on both client and server sides.
- Apply context-sensitive output encoding.
- Implement a strict Content Security Policy.
- Sanitize HTML using trusted libraries when HTML is required.
- Enable HttpOnly and Secure cookie flags appropriately.
- Perform regular XSS security testing.

3.4 Cross-Site Request Forgery

Vulnerability: Cross-Site Request Forgery (CSRF)

Affected URL: http://testfire.net/bank/main.jsp

Burp Suite Community Edition was configured as an intercepting proxy. The Intercept function was enabled to capture HTTP requests exchanged between the browser and the target application.

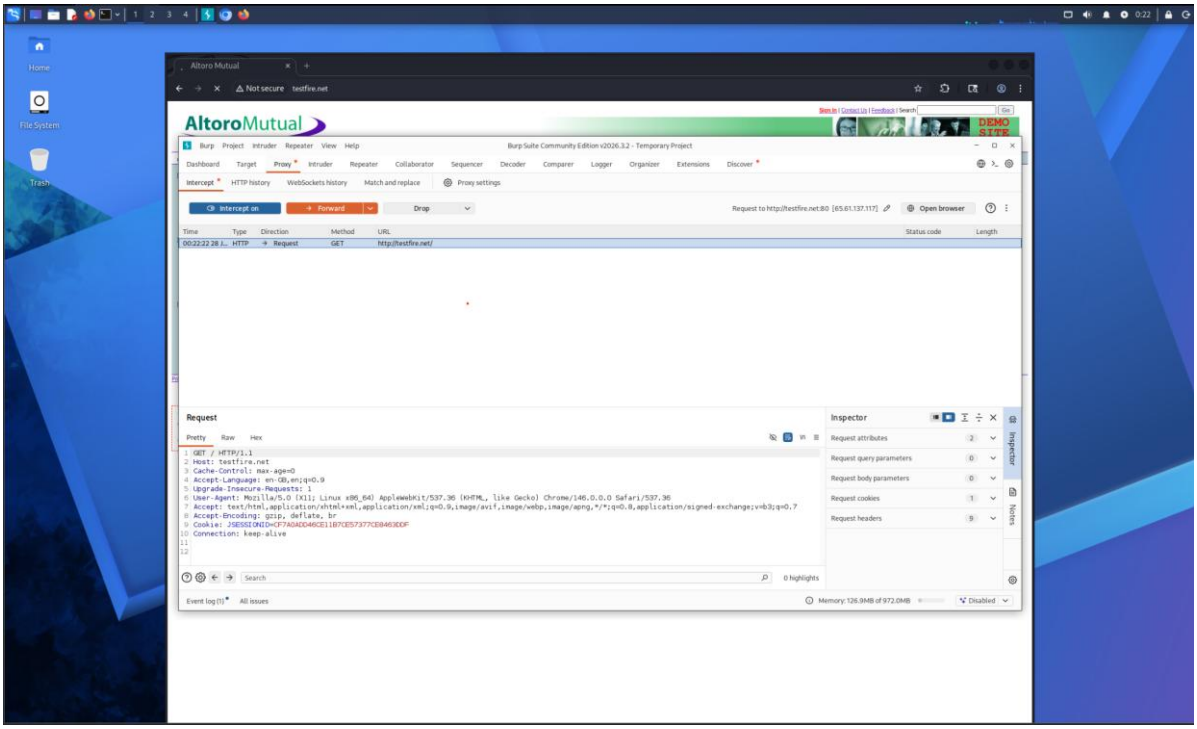

Figure 3.12 — Authenticated Application Session

Information / Observation: The screenshot shows the application accessed through the testing environment after authentication. The assessment used a valid user session for sensitive transaction testing.

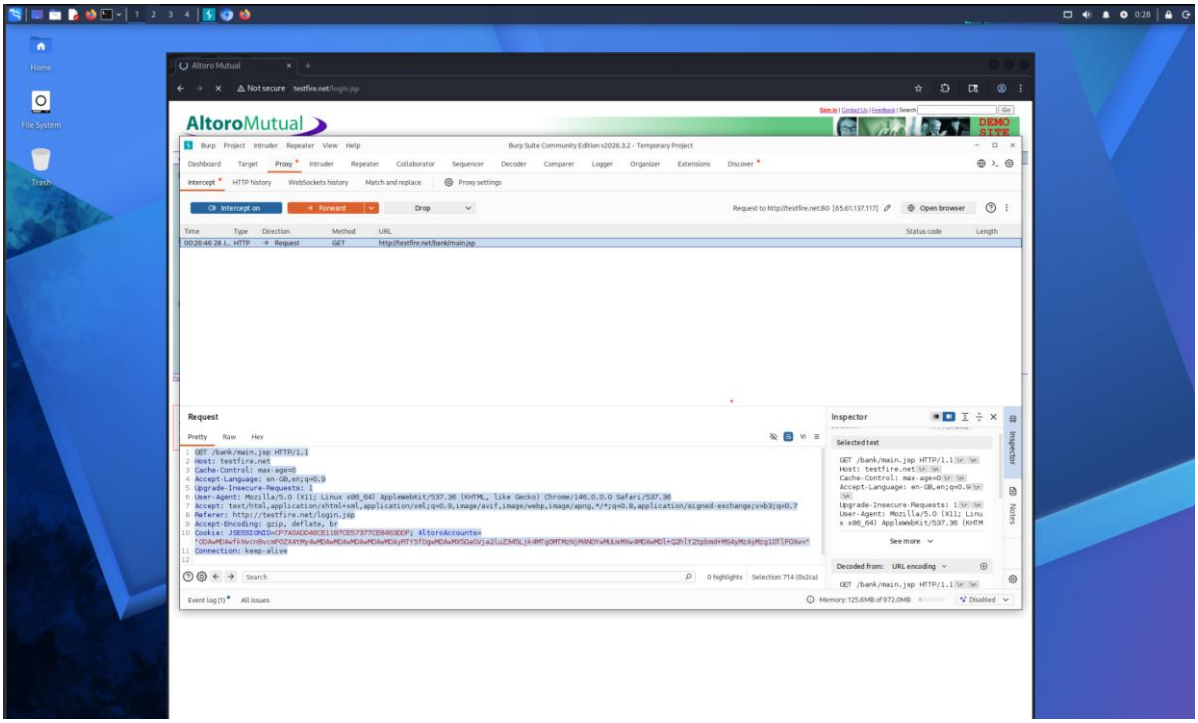

Figure 3.13 — Intercepted Authenticated Request

Information / Observation: The intercepted traffic demonstrates that the application maintains the authenticated session using a JSESSIONID cookie.

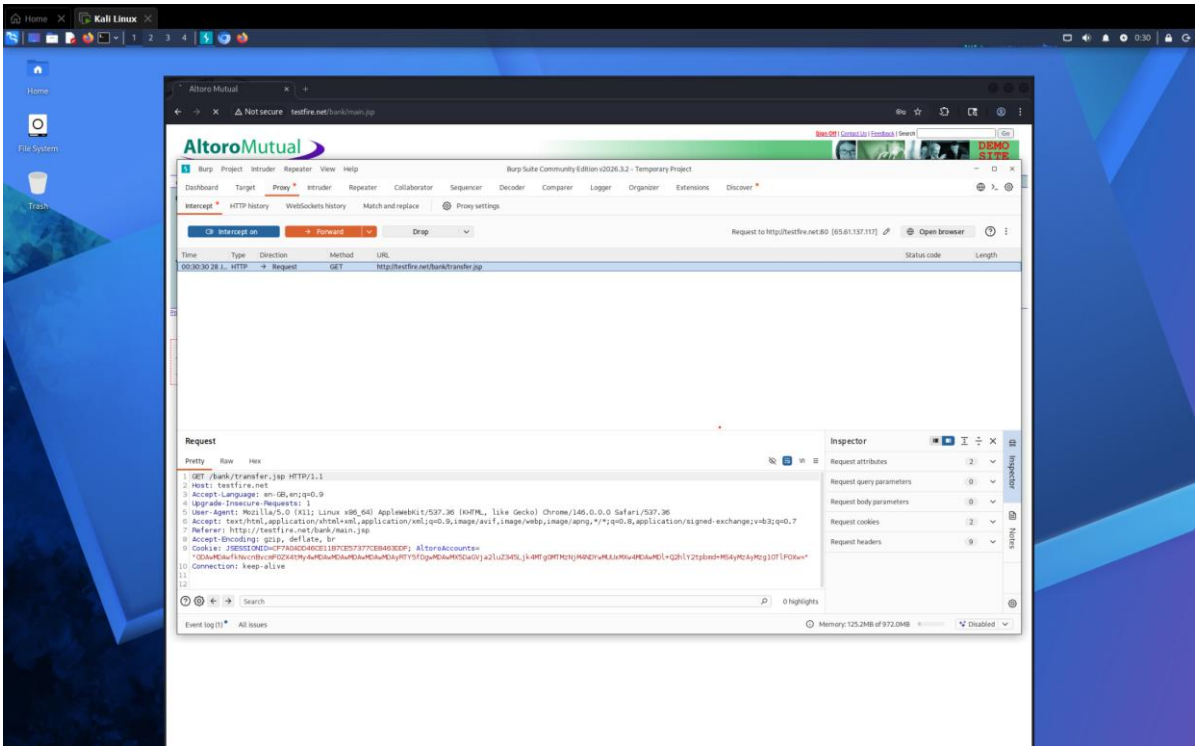

Figure 3.14 — Transfer Funds Request Capture

Information / Observation: The screenshot shows the sensitive Transfer Funds functionality selected for CSRF analysis. The outgoing request was captured for inspection.

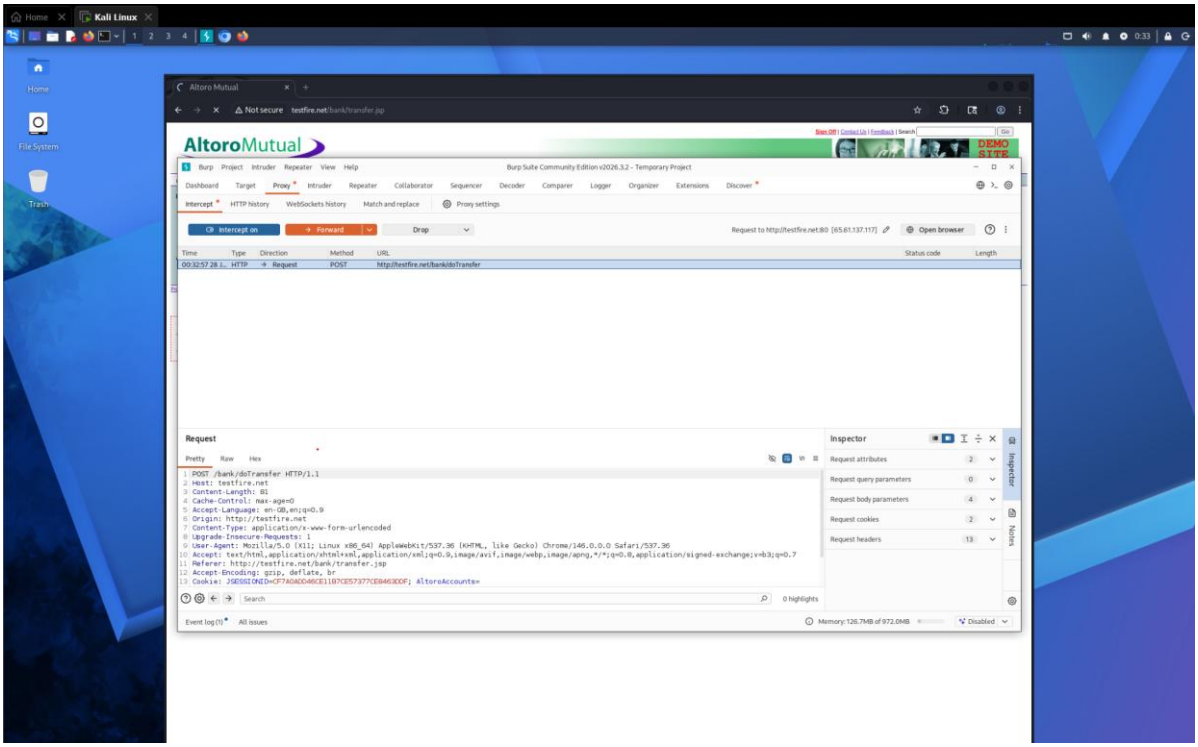

Figure 3.15 — CSRF Request Inspection

Information / Observation: The screenshot documents the request-analysis stage used to determine whether an anti-CSRF token was present.

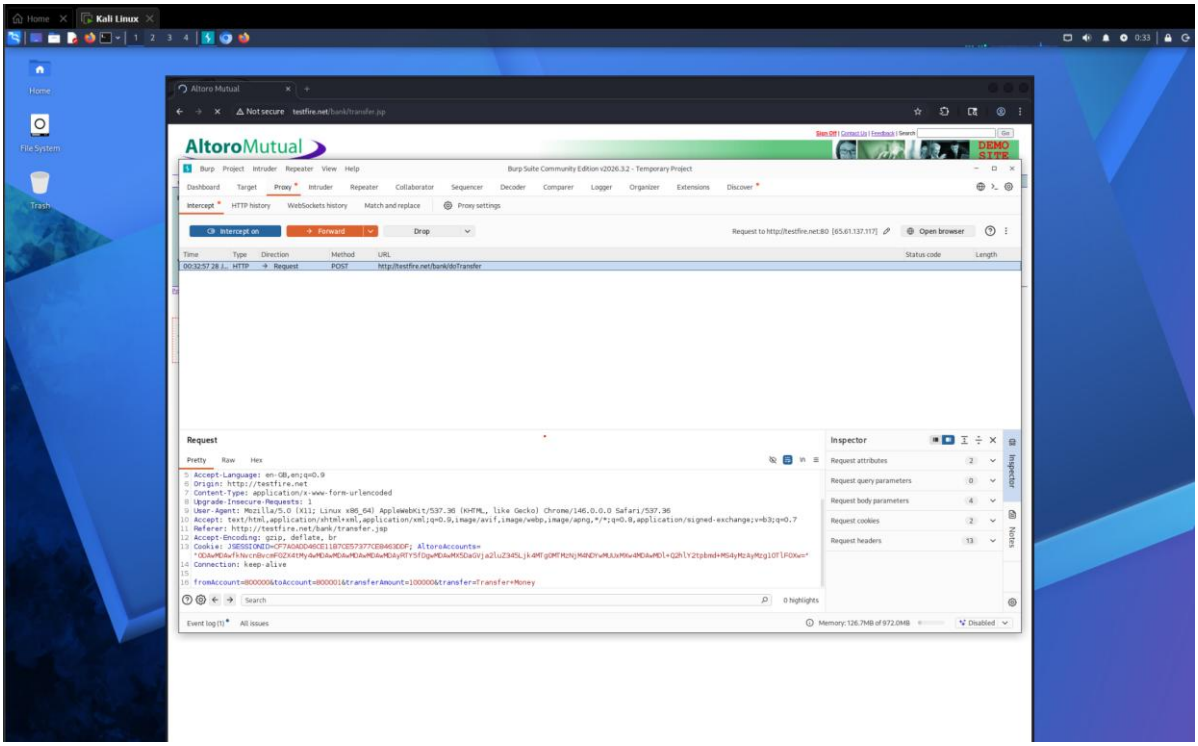

Figure 3.16 — CSRF Protection Observation

Information / Observation: The captured request did not show a visible anti-CSRF token. The finding therefore indicates a potential protection weakness requiring server-side validation of alternative controls.

Risk Level: Medium / Requires Further Validation.

Potential Impact: If server-side CSRF protection is absent, an attacker could potentially induce an authenticated user to initiate unauthorized fund transfers, modify account information, or perform sensitive operations.

- Implement unique anti-CSRF tokens for state-changing requests.
- Configure SameSite cookie attributes appropriately.
- Validate Origin and Referer headers where suitable.
- Perform server-side validation before processing sensitive transactions.
- Use additional transaction confirmation or re-authentication for high-risk operations.

3.5 Server-Side Request Forgery Assessment

Assessment Type: SSRF attack-surface assessment.

The application was accessed through Burp Suite to observe HTTP requests. During reconnaissance, Swagger API documentation was identified and available REST API endpoints were reviewed for parameters commonly associated with SSRF, including URL, URI, Host, Callback, Redirect, File Path, and External Resource Reference.

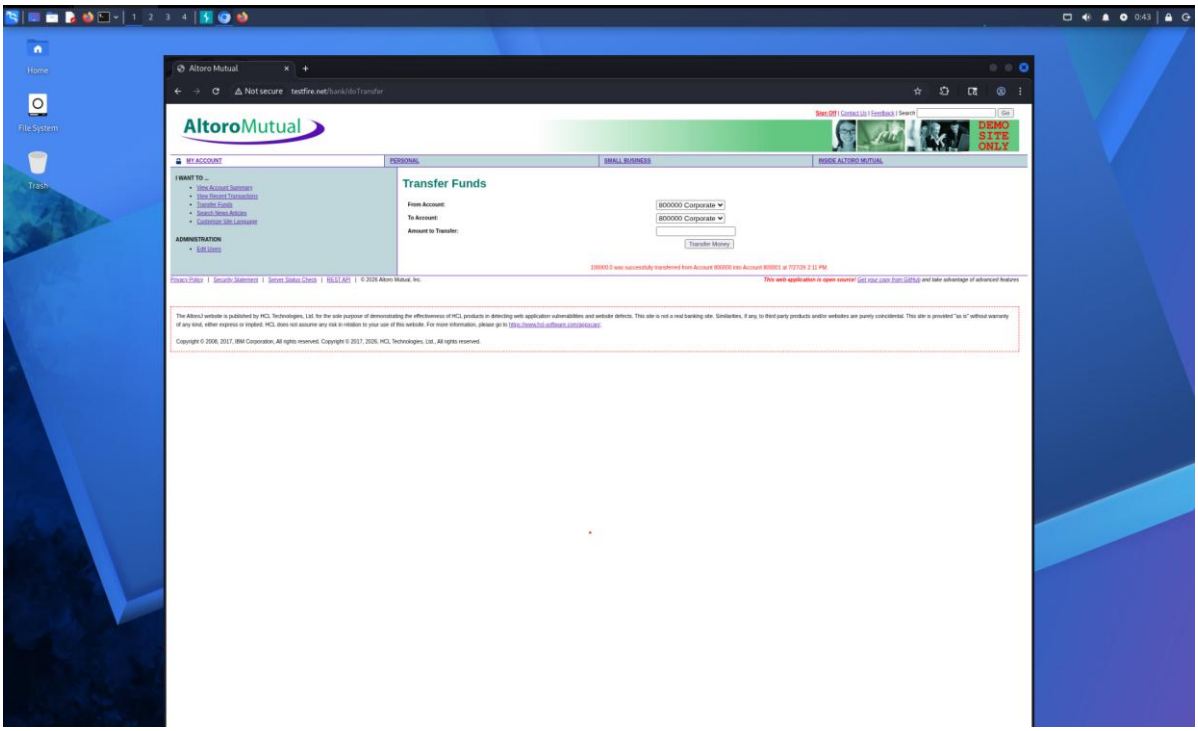

Figure 3.17 — Application/API Reconnaissance

Information / Observation: The screenshot documents the reconnaissance stage used to identify application modules and API-related functionality.

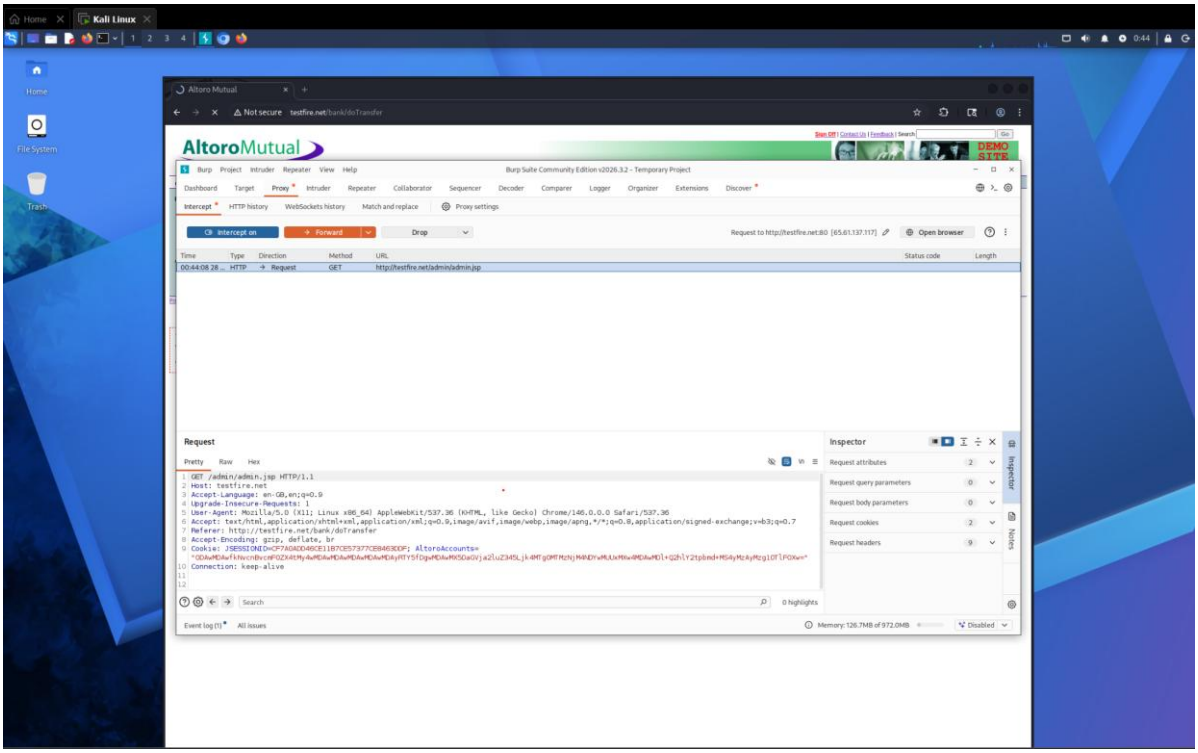

Figure 3.18 — Swagger/API Review

Information / Observation: The screenshot shows the API documentation identified during reconnaissance and reviewed for potential server-side request functionality.

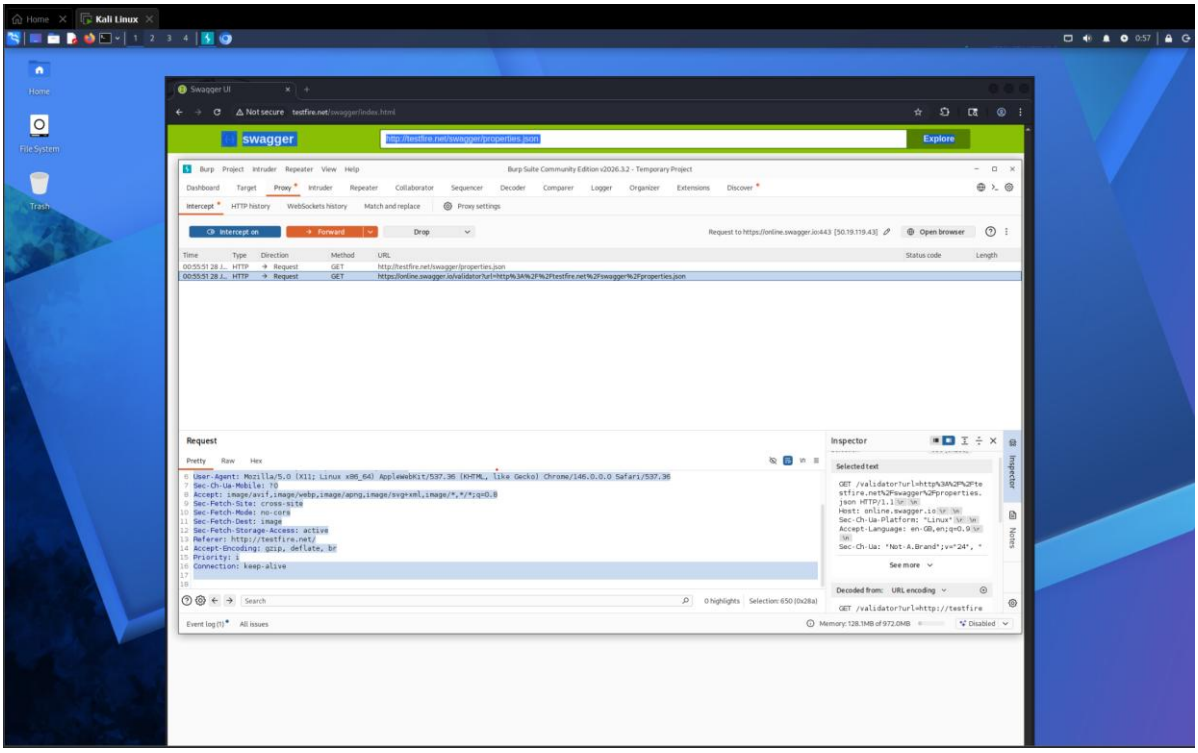

Figure 3.19 — API Request Analysis

Information / Observation: The screenshot documents the inspection of HTTP requests generated while reviewing the API documentation.

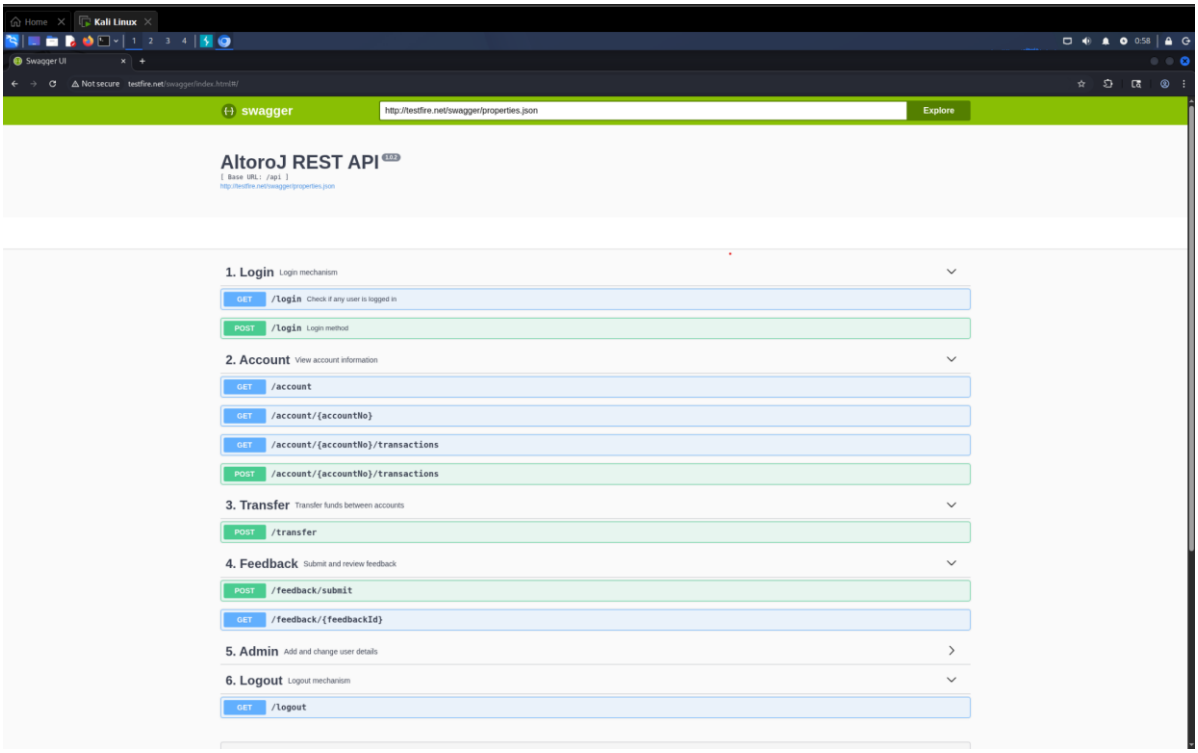

Figure 3.20 — API Endpoint Review

Information / Observation: The screenshot shows the available API endpoint view used to evaluate whether user-controlled parameters could cause server-side resource retrieval.

Observation: The documented assessment did not identify application functionality capable of initiating server-side requests using user-controlled input. No URL-fetching, webhook, remote-file-retrieval, or external-resource-loading feature suitable for SSRF exploitation was identified.

Result: No SSRF attack surface was identified during the documented assessment.

Risk Level: Informational – No vulnerability identified.

- Validate all user-supplied URLs.
- Use allowlists for trusted external domains.
- Block localhost and internal IP ranges.
- Restrict outbound network communication where possible.
- Review newly introduced API endpoints for SSRF risks.

3.6 SQL Injection – Authentication Testing

Objective: Determine whether the application's login functionality was vulnerable to SQL Injection and whether user-controlled authentication parameters were directly incorporated into backend SQL queries.

The login page accepts Username and Password inputs. The authentication functionality was selected for SQL Injection testing.

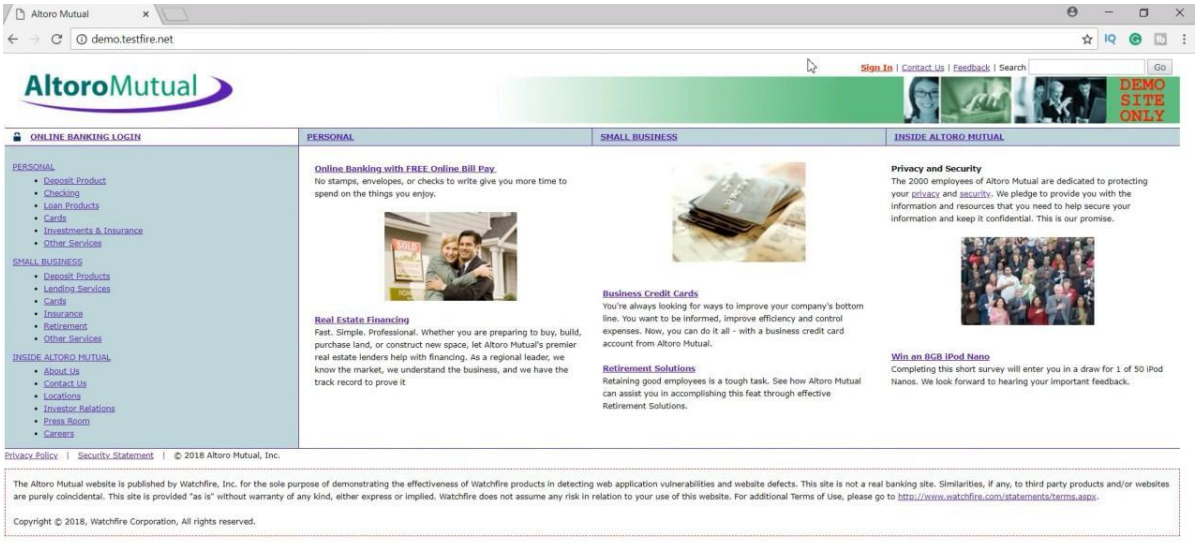

Figure 3.21 — Altoro Mutual Login Functionality

Information / Observation: The screenshot shows the target login functionality selected for SQL Injection and authentication testing.

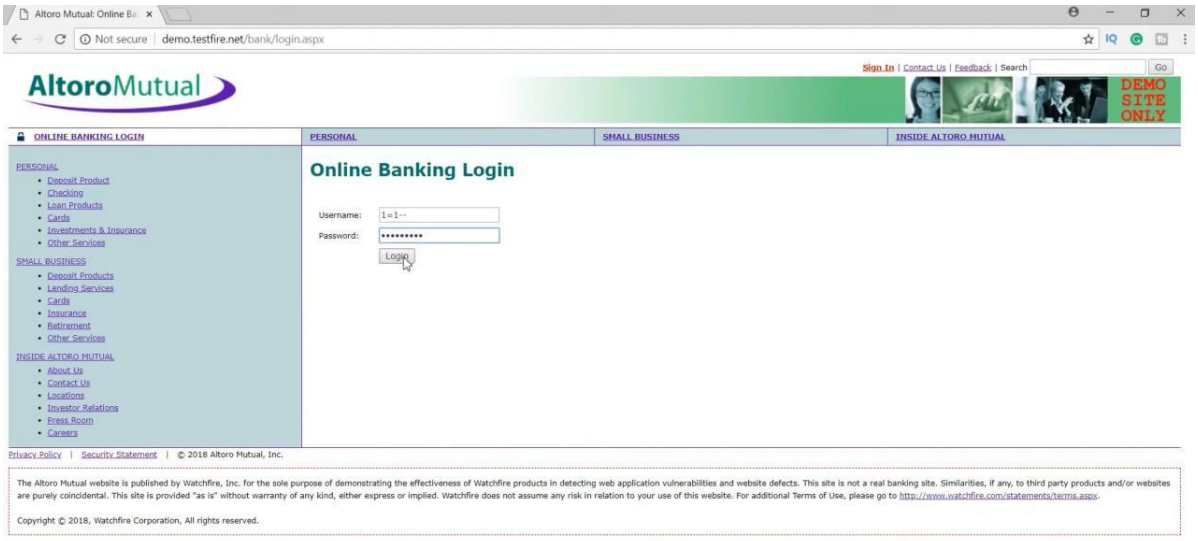

Figure 3.22 — SQL Injection Test Input

Information / Observation: The screenshot documents the first stage of SQL Injection testing against the authentication interface.

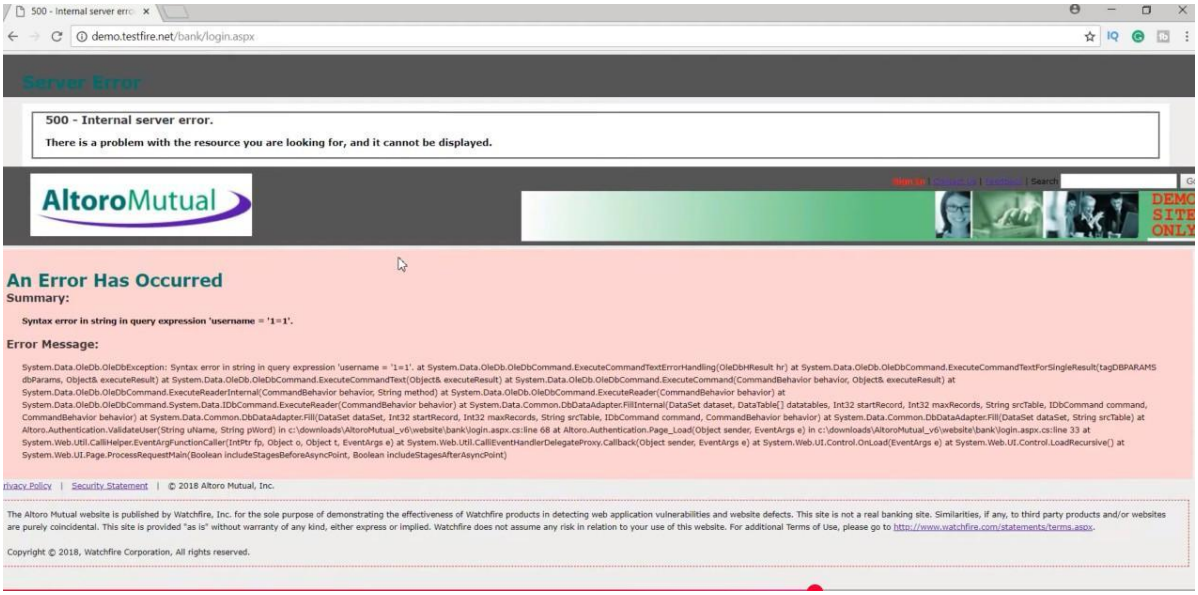

Figure 3.23 — SQL Error Response

Information / Observation: The application returned a server/database error after crafted input was supplied, indicating that user input was reaching query-processing logic.

Test Payload 1: '1'=1'

Observation: The application returned a 500 Internal Server Error along with a database exception, including the message “Syntax error in string in query expression”.

Security Impact: Improper input validation, SQL query construction exposure, database error disclosure, and potential SQL Injection vulnerability.

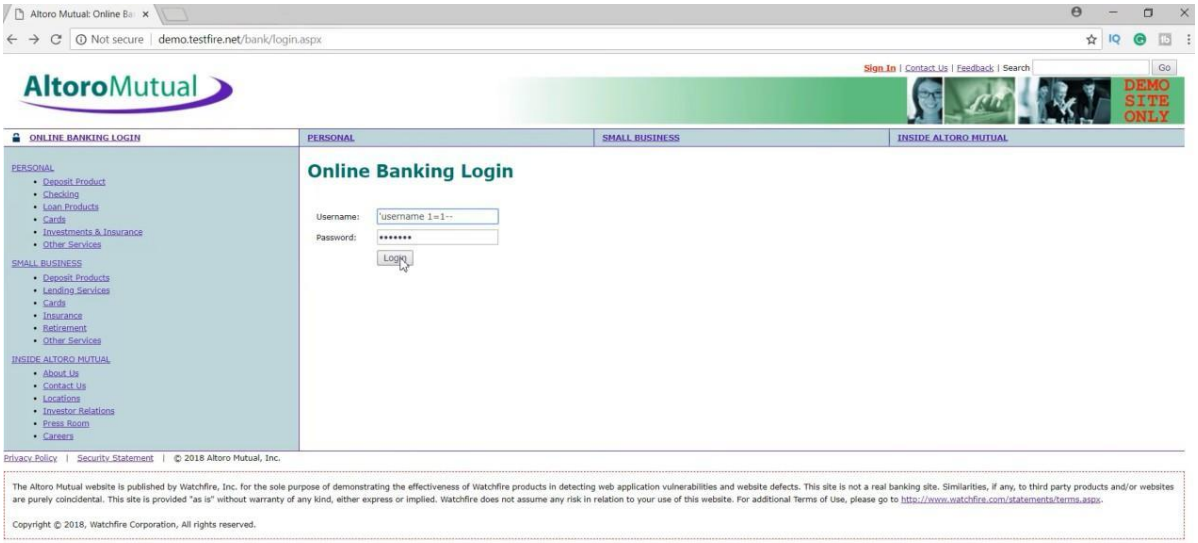

Figure 3.24 — SQL Syntax Manipulation Test

Information / Observation: The screenshot shows the second SQL manipulation test being performed against the authentication functionality.

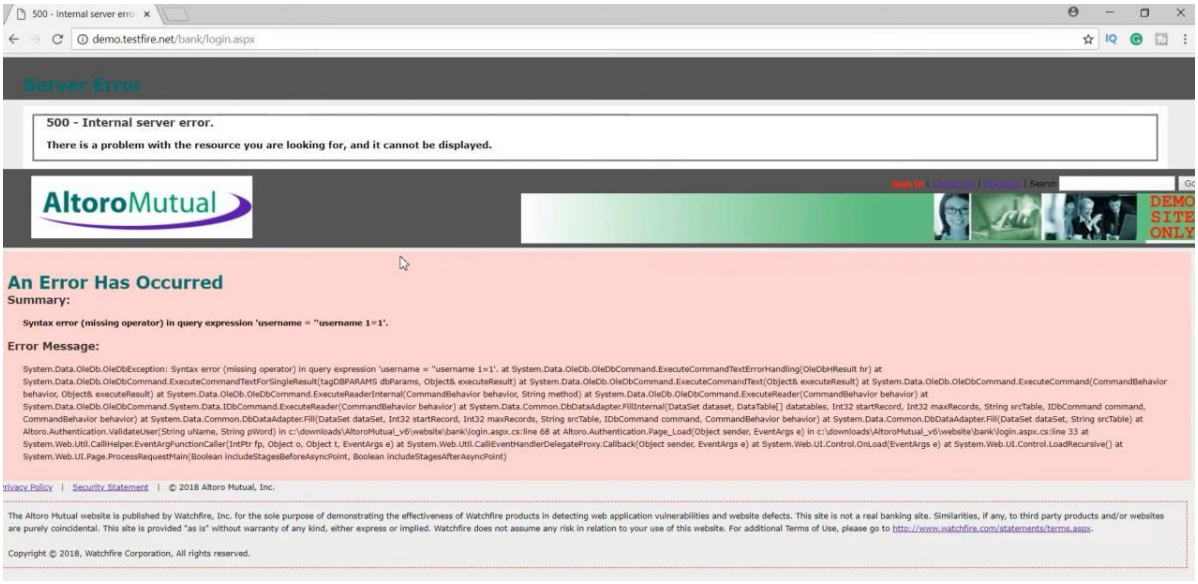

Figure 3.25 — SQL Syntax Error Result

Information / Observation: The screenshot shows the resulting application error. The documented response was “Syntax error (missing operator)”, further indicating that input was being interpreted as part of the SQL statement.

Test Payload 2: username 1=1--

Observation: The application again generated a 500 Internal Server Error and disclosed a SQL syntax-related error.

Security Impact: Backend query manipulation, error-based SQL Injection, and database implementation disclosure.

Risk Level: Critical.

3.7 SQL Injection – Authentication Bypass

Objective: Determine whether SQL Injection could bypass the authentication mechanism.

```
' OR 1=1--
```

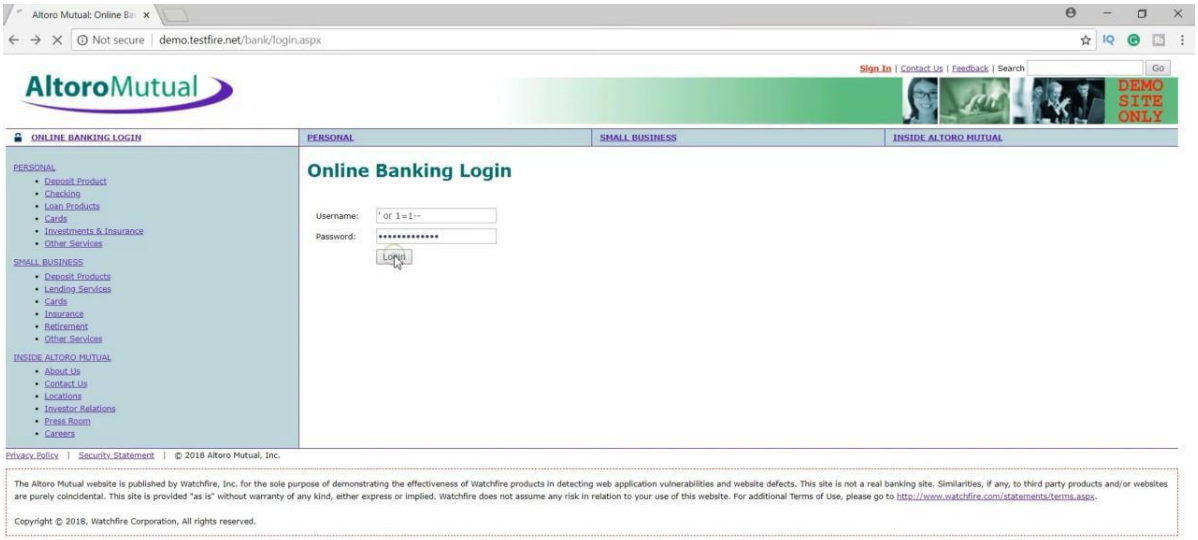

Figure 3.26 — Authentication Bypass Payload Test

Information / Observation: The screenshot documents the SQL Injection authentication-bypass test using the crafted condition intended to alter the authentication query logic.

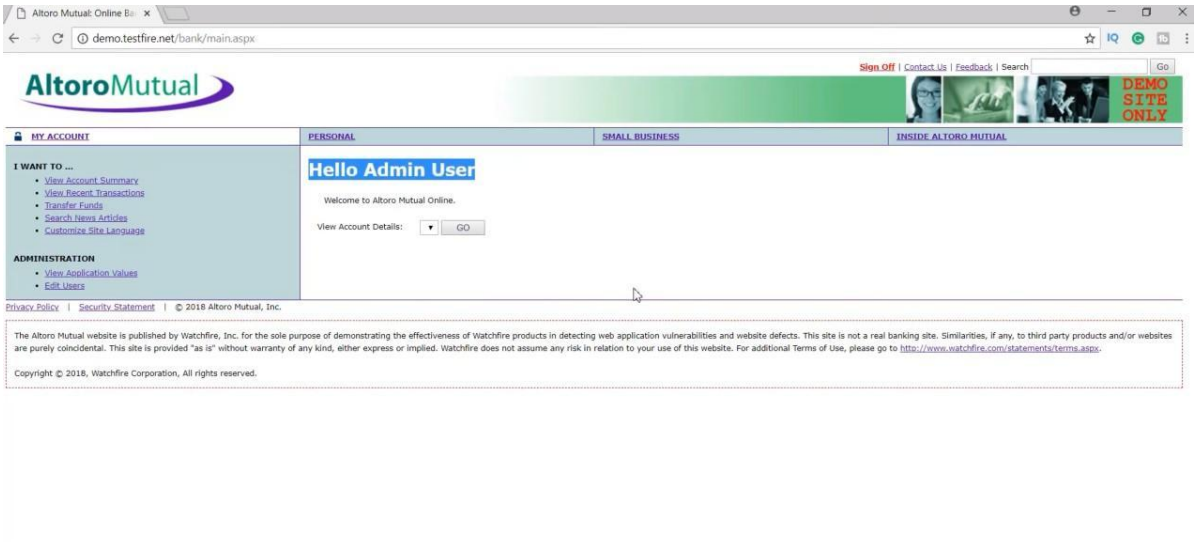

Figure 3.27 — Authentication Bypass Result

Information / Observation: The screenshot shows the application response associated with the authentication-bypass test. The documented project result states that authentication succeeded without valid credentials.

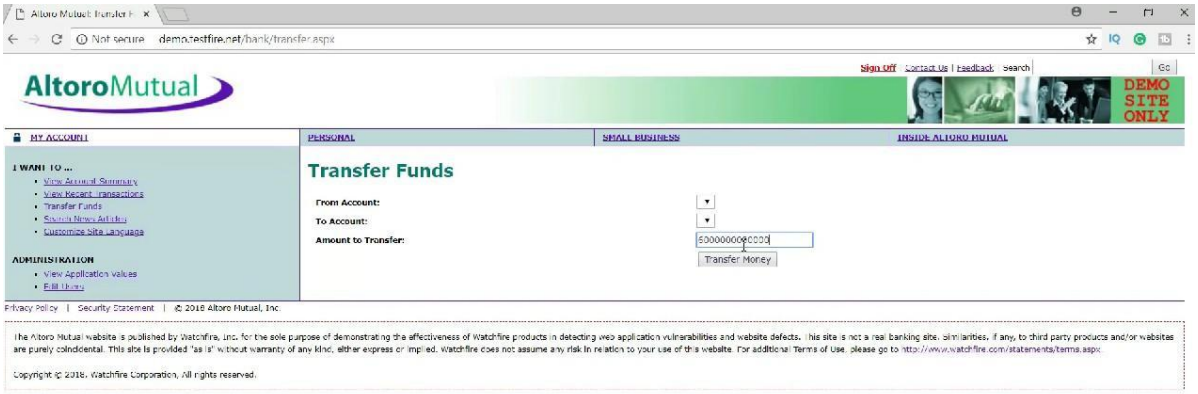

Figure 3.28 — Unauthorized Access to Privileged Functionality

Information / Observation: The screenshot demonstrates access to privileged banking functionality after the documented authentication bypass.

Result: Authentication Bypass Successful.

Security Impact: Login authentication was bypassed, unauthorized administrative access was obtained, and privileged banking functionality became accessible, including the Transfer Funds page.

Risk Level: Critical.

- Use parameterized queries and prepared statements.
- Never construct SQL queries using direct string concatenation.
- Validate user input on the server side.
- Do not expose detailed database errors to users.
- Use least-privilege database accounts.
- Implement secure authentication and authorization controls.
- Perform regular SQL Injection testing.
- Monitor and log suspicious authentication activity.

4. Vulnerability Risk Analysis & Evaluation

The assessment identified multiple application and network security weaknesses. The following matrix consolidates the documented findings and distinguishes confirmed vulnerabilities from findings requiring additional validation or negative assessments.

No.	Finding	Category	Risk / Status
1	Reflected XSS	Web Application	High
2	Stored XSS	Web Application	High
3	HTML Injection	Web Application	Medium
4	CSRF Protection Weakness	Web Application	Medium / Further Validation
5	SSRF Attack Surface	Web Application	Not Identified
6	SQL Injection	Injection	Critical
7	SQL Injection Authentication Bypass	Authentication / Injection	Critical
8	Host Discovery	Network Reconnaissance	Informational
9	OS Detection	Information Disclosure	Low
10	Service/Version Disclosure	Information Disclosure	Medium
11	TLS 1.0 / TLS 1.1 Enabled	Cryptographic Configuration	Medium
12	Weak DH1024	Cryptographic Configuration	Medium
13	CBC Cipher Suites	Cryptographic Configuration	Medium

4.1 Consolidated Vulnerability Summary

The highest-priority findings are the SQL Injection and authentication-bypass issues because the documented testing demonstrated unauthorized authentication and access to privileged banking functionality. Stored and reflected XSS were also confirmed and require prompt remediation. Network and TLS findings should be addressed as part of server and infrastructure hardening.

4.2 Impact Analysis

SQL Injection: The authentication-bypass test demonstrated the most serious impact. Successful manipulation of authentication logic can result in unauthorized account access and exposure of privileged functions.

Cross-Site Scripting: XSS can enable malicious JavaScript execution in victim browsers and may result in session compromise, phishing, information disclosure, page manipulation, and client-side attacks.

HTML Injection: Attacker-controlled markup can alter trusted page content and support phishing or malicious redirection.

CSRF: If effective server-side protection is absent, authenticated users could potentially be induced to perform unauthorized state-changing operations.

Network Misconfiguration: Exposed services and technology information assist attackers during reconnaissance and vulnerability identification.

Weak TLS Configuration: Legacy protocols and weak cryptographic parameters reduce the security of encrypted communications.

4.3 Security Risk Evaluation

Overall, the assessment indicates significant security weaknesses at the input-validation and authentication layers. Priority should be assigned to SQL Injection, authentication bypass, Stored XSS, Reflected XSS, TLS hardening, service/version information disclosure, and validation of CSRF controls. The SSRF assessment did not identify an exploitable attack surface in the documented test.

5. Recommendations & Security Hardening

5.1 Secure Database Interaction

- Use prepared statements and parameterized queries.
- Avoid dynamic SQL string concatenation.
- Apply strict server-side input validation.
- Use least-privilege database accounts.
- Suppress database errors from end users.

5.2 XSS Protection

- Perform server-side input validation.
- Implement context-aware output encoding.
- Sanitize HTML where required.
- Implement Content Security Policy.
- Configure secure session cookies.
- Use HttpOnly and Secure flags appropriately.

5.3 Authentication Security

- Prevent SQL Injection in login functionality.
- Implement strong authentication controls.
- Apply rate limiting against repeated authentication attempts.
- Monitor authentication failures.
- Avoid detailed authentication error messages.
- Consider multi-factor authentication for sensitive accounts.

5.4 CSRF Protection

- Implement anti-CSRF tokens.
- Validate Origin and Referer headers where appropriate.
- Configure SameSite cookie attributes.
- Require additional confirmation for high-risk financial transactions.

5.5 Network Hardening

- Close unnecessary ports.
- Restrict administrative services.
- Use firewall rules and network segmentation.
- Monitor scanning activity.
- Keep server software updated.

5.6 Server Information Disclosure

- Disable unnecessary server banners.
- Remove unnecessary HTTP headers.
- Avoid exposing software versions.
- Keep Apache Tomcat and related components patched.

5.7 TLS Hardening

- Disable TLS 1.0 and TLS 1.1.
- Enable TLS 1.2 and TLS 1.3.
- Replace weak DH1024 parameters.
- Prefer modern ECDHE key exchange.
- Disable weak cipher suites where possible.
- Use trusted certificates in production environments.

5.8 Continuous Security Testing

- Periodic vulnerability scanning
- Regular penetration testing
- Secure code review
- Dependency vulnerability monitoring
- Security regression testing
- Continuous security monitoring
- Incident-response procedures

6. Conclusion

This major project successfully demonstrated a structured Vulnerability Assessment and Penetration Testing methodology against the Altoro Mutual demonstration banking application hosted on testfire.net.

The assessment combined network-level and application-level security testing using Kali Linux, Burp Suite, Nmap, and SQLmap. The reconnaissance phase identified an active target system and exposed services including HTTP, HTTPS, and HTTP Alternate services. Service enumeration identified Apache Tomcat and Apache Coyote JSP Engine information. The SSL/TLS assessment identified legacy TLS protocols and weak cryptographic parameters requiring hardening.

The web application assessment identified multiple vulnerabilities. Reflected XSS was demonstrated through the search functionality, while Stored XSS was identified in the feedback functionality. HTML Injection was also demonstrated through the search functionality.

The CSRF assessment identified the absence of a visible anti-CSRF token in the captured fund-transfer request. Because the documented test did not establish whether alternative server-side controls existed, this finding should be subjected to additional validation.

The SSRF assessment did not identify application functionality capable of initiating server-side requests based on user-controlled input. Therefore, no SSRF vulnerability was confirmed during the documented assessment.

The most significant finding was SQL Injection in the authentication functionality. Database errors were generated when crafted input was submitted, indicating inadequate input handling. More importantly, the documented authentication-bypass payload successfully bypassed authentication and provided access to privileged banking functionality.

The project demonstrates that improper input validation, insecure query construction, insufficient client-side security controls, weak authentication mechanisms, excessive information disclosure, and outdated cryptographic configurations can significantly increase the attack surface of a web application.

A layered security approach, continuous assessment, secure development practices, timely patching, strong authentication, secure coding, and appropriate network controls are essential for reducing application and infrastructure security risk.

7. References & Standards

25. OWASP Foundation — OWASP Web Security Testing Guide (WSTG), guidance for web application security testing.
26. OWASP Foundation — OWASP Top 10 Web Application Security Risks.
27. PortSwigger — Burp Suite documentation and Web Security Academy resources.
28. Nmap Project — Nmap Network Scanning and Reference Documentation.
29. sqlmap Project — SQL Injection detection and validation documentation.
30. C-DAC — Cybersecurity and network security testing laboratory resources.
31. RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3.
32. RFC 5246 — The Transport Layer Security (TLS) Protocol Version 1.2.
33. OWASP Cross-Site Scripting Prevention Cheat Sheet.
34. OWASP SQL Injection Prevention Cheat Sheet.
35. OWASP Cross-Site Request Forgery Prevention Cheat Sheet.
36. NIST — Technical guidance related to vulnerability assessment and information security testing.
37. Altoro Mutual / testfire.net — Demonstration web application used as the controlled target environment for the security assessment.

Appendix A: Screenshot Evidence Index

The following evidence images were extracted from the submitted Major Project document and incorporated into this expanded report. Each image is followed by contextual information describing what the screenshot demonstrates.

Evidence File	Description
p1_1.png	Reflected XSS – Search functionality
p2_1.png	Reflected XSS – Payload submission
p2_2.png	Reflected XSS – Execution result
p3_1.png	Cookie access test
p3_2.png	Cookie test response
p4_1.png	HTML Injection – Search test
p5_1.png	HTML Injection – Result
p6_1.png	Stored XSS – Feedback testing
p6_2.png	Stored XSS – Burp request analysis
p7_1.png	Stored XSS – Payload
p7_2.png	Stored XSS – Result
p9_1.png	Authenticated application session
p9_2.png	Intercepted authenticated request
p10_1.png	Transfer Funds request
p11_1.png	CSRF request inspection
p11_2.png	CSRF protection observation
p13_1.png	Application/API reconnaissance
p14_1.png	Swagger/API review
p15_1.png	API request analysis
p16_1.png	API endpoint review
p18_1.jpeg	Altoro Mutual login
p19_1.jpeg	SQL Injection test input
p19_2.jpeg	SQL error response
p21_1.jpeg	SQL syntax manipulation
p21_2.jpeg	SQL syntax error result
p23_1.jpeg	Authentication bypass payload
p23_2.jpeg	Authentication bypass result
p24_1.jpeg	Unauthorized privileged functionality
p25_1.jpeg	Network/host discovery evidence
p25_2.png	Nmap host discovery
p27_1.png	Nmap OS detection
p29_1.png	Nmap service/version detection

p31_1.png	Nmap SSL/TLS enumeration